

HW4_Report



Author: 111511198 鄭恆安

Concept and Code Details of KNN Model

The **K-Nearest Neighbors (KNN)** algorithm is a non-parametric, instance-based learning method. It classifies a data point based on the majority class of its k -nearest neighbors in the feature space. The proximity is determined by a distance metric such as Euclidean, Manhattan, or Chebyshev distance.

Key Concepts in the Code:

1. Distance Metrics:

- The `_cal_distance` method computes the distance between two points. The choice of metric (`euclidean` , `manhattan` , or `chebyshev`) affects how neighbors are selected.

```
def _cal_distance(self, x1, x2):
    distance = 0
    for i in range(len(x1)):
        x1[i] = float(x1[i])
        x2[i] = float(x2[i])
        if(self.distance_metric == 'euclidean'):
            distance += (x1[i] - x2[i])**2
        elif(self.distance_metric == 'manhattan'):
            distance += abs(x1[i] - x2[i])
        elif(self.distance_metric == 'chebyshev'):
            distance = max(distance, abs(x1[i] - x2[i]))
    if(self.distance_metric == 'euclidean'):
        return np.sqrt(distance)
    else:
        return distance
```

2. Fit Method:

- Simply stores the training data (`X_train` and `y_train`). No actual model training occurs in KNN, as it is lazy learning.

```
def fit(self, X, y):
    if(isinstance(X, pd.DataFrame)):
        X = X.values
        y = y.values
    self.X_train = X
    self.y_train = y
```

```
self.weight_0 = np.sum(y == 1) / len(y)
self.weight_1 = np.sum(y == 0) / len(y)
```

3. Predict Method:

- For each test instance, `_predict` calculates distances to all training instances and selects the k-nearest ones.
- The majority class among these neighbors determines the predicted label.

```
def predict(self, X):
    if(isinstance(X, pd.DataFrame)):
        X = X.values
    y_pred = [self._predict(x) for x in X]
    return np.array(y_pred)
def _predict(self, x):
    distances = []
    for x_train in self.X_train:
        distance = self._cal_distance(x, x_train)
        distances.append(distance)
    k_indices = np.argsort(distances)[:self.k]
    label=0
    for i in range(len(k_indices)):
        if(self.y_train[k_indices[i]] == 0):
            label+=self.weight_0
            #label+=1
        else:
            label-=self.weight_1
            #label-=1
    if(label>0):
        return 0
    else:
        return 1
```

4. Metrics for Evaluation:

- `predict_score` computes accuracy, F1-score, and MCC, combining them into a custom scoring metric.

```
def predict_score(self, pred, y):
    acc = accuracy_score(pred, y)
    f1 = f1_score(pred, y, zero_division=0)
    mcc = matthews_corrcoef(pred, y)
    scoring = 0.3 * acc + 0.35 * f1 + 0.35 * mcc

    return scoring
```

5. KFold_cross_validation:

- Splits the dataset into n folds.
- Iteratively uses one fold for validation and the remaining n-1 folds for training.
- Evaluates the model using three metrics: **Accuracy**, **F1-Score**, and **Matthews Correlation Coefficient (MCC)**.
- Returns the averaged metrics and a combined scoring value.

```
def KFold_cross_validation(X, y, n, k=3, method='euclidean'):
    fold_size = len(X) // n
    data = pd.concat([X, y], axis=1)
    data = data.sample(frac=1).reset_index(drop=True)

    scores = {
        'accuracy': [],
        'f1': [],
        'mcc': [],
    }
    print('k:', k, 'method:', method)
    for i in tqdm(range(n)):
        start, end = i * fold_size, (i + 1) * fold_size
        val_data = data[start:end]
        train_data = pd.concat([data[:start], data[end:]])

        X_train, y_train = train_data.iloc[:, :-1], pd.DataFrame(train_data
X_val, y_val = val_data.iloc[:, :-1], val_data.iloc[:, -1]

        X_train = X_train.values
        y_train = y_train.values
        X_val = X_val.values
        y_val = y_val.values

        model = KNearestNeighborClassifier(k, method)
        model.fit(X_train, y_train)
        y_pred = model.predict(X_val)

        scores['accuracy'].append(accuracy_score(y_val, y_pred))
        scores['f1'].append(f1_score(y_val, y_pred))
        scores['mcc'].append(matthews_corrcoef(y_val, y_pred))

    for metric in scores:
        scores[metric] = np.mean(scores[metric])
    scoring = 0.3 * scores['accuracy'] + 0.35 * scores['f1'] + 0.35 * scor
    scores['scoring'] = scoring
    print('scoring:', scoring)
    print('\n')
    return scores
```

6. Find_best_params:

- Iterates over a range of k values and distance metrics.
- Performs K-Fold Cross-Validation for each combination of k and the metric.
- Selects the combination with the highest scoring value.

```
def find_best_params(X, y, params):
    best_params = {
        'k': 0,
        'method': ''
    }
    best_score = 0
    for method in params['method']:
        k_values = []
        scores_list = []
        for k in params['k']:
            scores = KFold_cross_validation(X, y, 5, k, method)
            if scores['scoring'] > best_score:
                best_score = scores['scoring']
                best_params['k'] = k
                best_params['method'] = method
            k_values.append(k)
            scores_list.append(scores['scoring'])
        print('best_params:', best_params, 'best_score:', best_score)
    return best_params, best_score
```

Questions

(a) If the feature size is too large and redundant, how will you select the features?

solution:

1. Compute Feature Correlations:

- Use a correlation matrix (e.g., Pearson or Spearman correlation) to calculate the correlation between all pairs of features.

2. Set a Threshold:

- Define a threshold (e.g., 0.11) to identify pairs of features that are highly correlated.
- Features with correlation lower than this threshold are considered redundant.

3. Feature Elimination:

- For each pair of highly correlated features, retain one and eliminate the other. This can be done based on:
 - Importance to the target variable (e.g., higher correlation with the label).

- Theoretical significance or interpretability.

4. Evaluate Performance:

- After feature selection, evaluate the KNN model's performance on a validation set.
- Experiment with different thresholds and find out that 0.11 to be optimal.

```

"""preprocessor.py"""
def feature_selection(self, threshold):
    # Return the correlation between features
    # find absolute correlation >= threshold
    column = self.df.corr()['label'][abs(self.df.corr()['label']) >= thre
    # get the column name
    column = column.tolist()
    return column

"""main.py"""
# use feature_selection to select the feature

train_data = pd.concat([train_X, train_y], axis=1)
train_data = train_data.fillna(train_data.mean()) # 0 here stands for np.mean
selected_column = Preprocessor(train_data).feature_selection(threshold)

train_data = train_data[selected_column]

```

(b) Handling Data Imbalance

1. Resampling Techniques (Not used in this project):

- **Oversampling** the minority class, such as using SMOTE (Synthetic Minority Oversampling Technique), generates synthetic samples for the minority class to balance the dataset.
- **Undersampling** the majority class reduces the number of samples in the majority class, which might risk losing useful information.

2. Class Weighting Approach (Used in this project):

- This project uses a **weighted KNN approach**, where the contributions of neighbors are adjusted based on the class distribution.
- The weights are proportional to the **inverse of the class ratio** to penalize the majority class and give higher importance to the minority class.

Implementation Details:

In the `_predict` function, class weights are incorporated into the vote counting logic:

- **Class 0** contributes `weight_0` for each of its neighbors in the top k neighbors, where `weight_0` is inversely proportional to the size of class 1.
- **Class 1** contributes `weight_1` for each of its neighbors in the top k neighbors, where `weight_1` is inversely proportional to the size of class 0.

```

def fit(self, X, y):
    if(isinstance(X, pd.DataFrame)):
        X = X.values
        y = y.values
    self.X_train = X
    self.y_train = y
    self.weight_0 = np.sum(y == 1) / len(y)
    self.weight_1 = np.sum(y == 0) / len(y)
def _predict(self, x):
    distances = []
    for x_train in self.X_train:
        distance = self._cal_distace(x, x_train)
        distances.append(distance)
    k_indices = np.argsort(distances)[:self.k]
    label=0
    for i in range(len(k_indices)):
        if(self.y_train[k_indices[i]] == 0):
            label+=self.weight_0
            #label+=1
        else:
            label-=self.weight_1
            #label-=1
    if(label>0):
        return 0
    else:
        return 1

```

Curve of K and Error Rate

How to Find the Best Value of k for KNN?

1. Data Splitting:

- The dataset was split into **80% training data** and **20% validation data** to evaluate model performance on unseen data.

2. Range of k:

- k values were iterated from **1 to 40**, allowing an exploration of the relationship between k and the **validation error rate** using different distance metrics (Manhattan, Euclidean, and Chebyshev).

3. Observations:

- For **Manhattan Distance**:
 - The error rate is relatively low when k is small, but begins to stabilize and reach a minimum around k=35.
- For **Euclidean Distance**:

- The error rate fluctuates more but decreases after $k=10$, showing better stability between $k=30$ to $k=35$.
- For **Chebyshev Distance**:
 - The error rate initially increases significantly and stabilizes after $k=20$, but it does not outperform the Manhattan or Euclidean metrics.

4. Best Value of k :

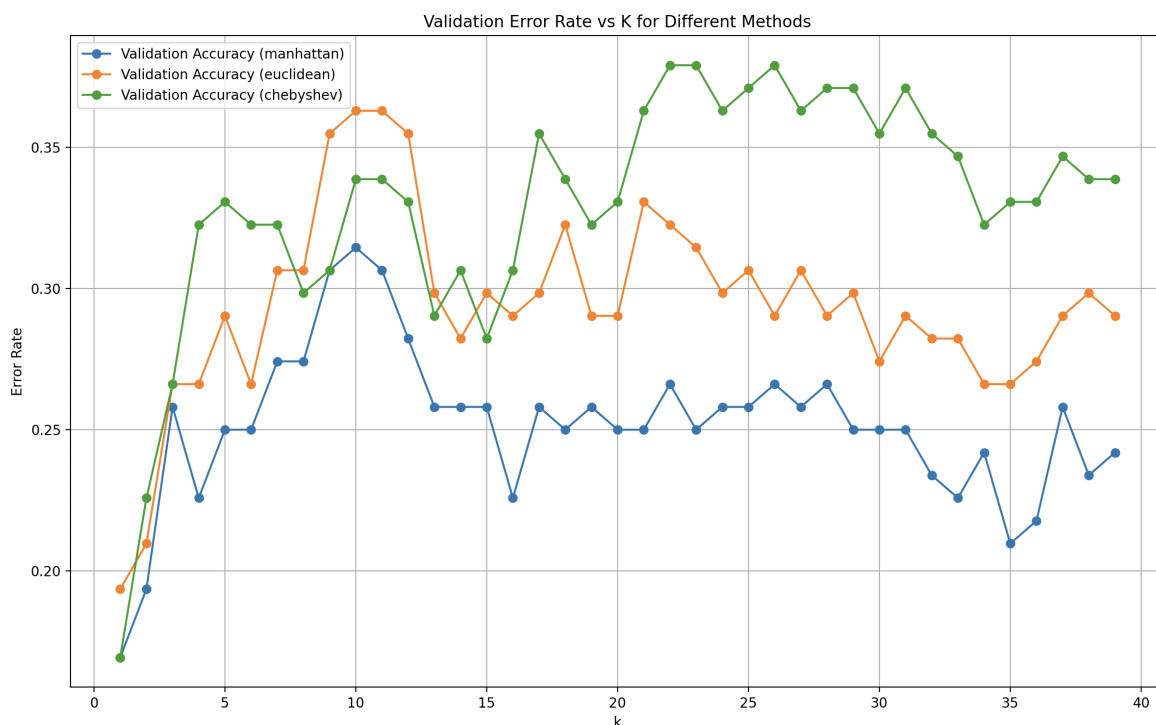
- Based on the graph:
 - For **Manhattan Distance**, the best k is around **35** due to the lowest error rate.
 - For **Euclidean Distance**, the best k is between 30 and 25, showing consistent performance.
 - For **Chebyshev Distance**, it does not perform as well as the other two metrics, and error rates remain relatively high.

5. Underfitting and Overfitting Trends:

- **Underfitting** occurs at small k values (e.g., $k < 10$) because the model becomes too sensitive to noise.
- As k increases, the model generalizes better, but a very high k can lead to **over-smoothing**, where local patterns are ignored.

6. Final Decision:

- The optimal k depends on the distance metric used:
 - **Manhattan Distance**: $k=35$
 - **Euclidean Distance**: $k=30-35$
 - **Chebyshev Distance**: Not recommended based on this graph.



```
def draw_relationship(train_X, train_y, val_X, val_y, k_values):
    if(isinstance(train_X, pd.DataFrame)):
        train_X = train_X.values
        train_y = train_y.values
        val_X = val_X.values
        val_y = val_y.values
    methods = ['manhattan', 'euclidean', 'chebyshev']
    for method in methods:
        val_scores = []
        for k in tqdm(k_values):
            model = KNearestNeighborClassifier(k=k, distance_metric=method)
            model.fit(train_X, train_y)
            val_pred = model.predict(val_X)
            val_score = accuracy_score(val_y, val_pred)
            val_error_rate = 1 - val_score
            val_scores.append(val_error_rate)
        plt.plot(k_values, val_scores, label=f'Validation Accuracy ({method})')
    plt.xlabel('k')
    plt.ylabel('Error Rate')
    plt.title('Validation Error Rate vs K for Different Methods')
    plt.legend()
    plt.grid(True)
    plt.show()
```

D. Discussion

(a) Finding the Best K and Method:

- Perform cross-validation on the training set, splitting it into train/validation subsets.
- Select parameters finding from the plot, K will be around 30 to 35
- Find the best_params

```
best_params: {'k': 31, 'method': 'manhattan'}
best_score: 0.6159134544413419 # average validation score
```

(b) Comparison of Distance Metrics:

- Use a validation set to evaluate performance (accuracy, F1-score, MCC) for each distance metric (`euclidean`, `manhattan`, `chebyshev`).
- Manhattan distance have better performance on this dataset.

```
Method: manhattan K: 31
Scoring: 0.66346
```

```
Method: euclidean K: 32
```


Scoring: 0.61974

Method: chebyshev K: 35

Scoring: 0.55211
